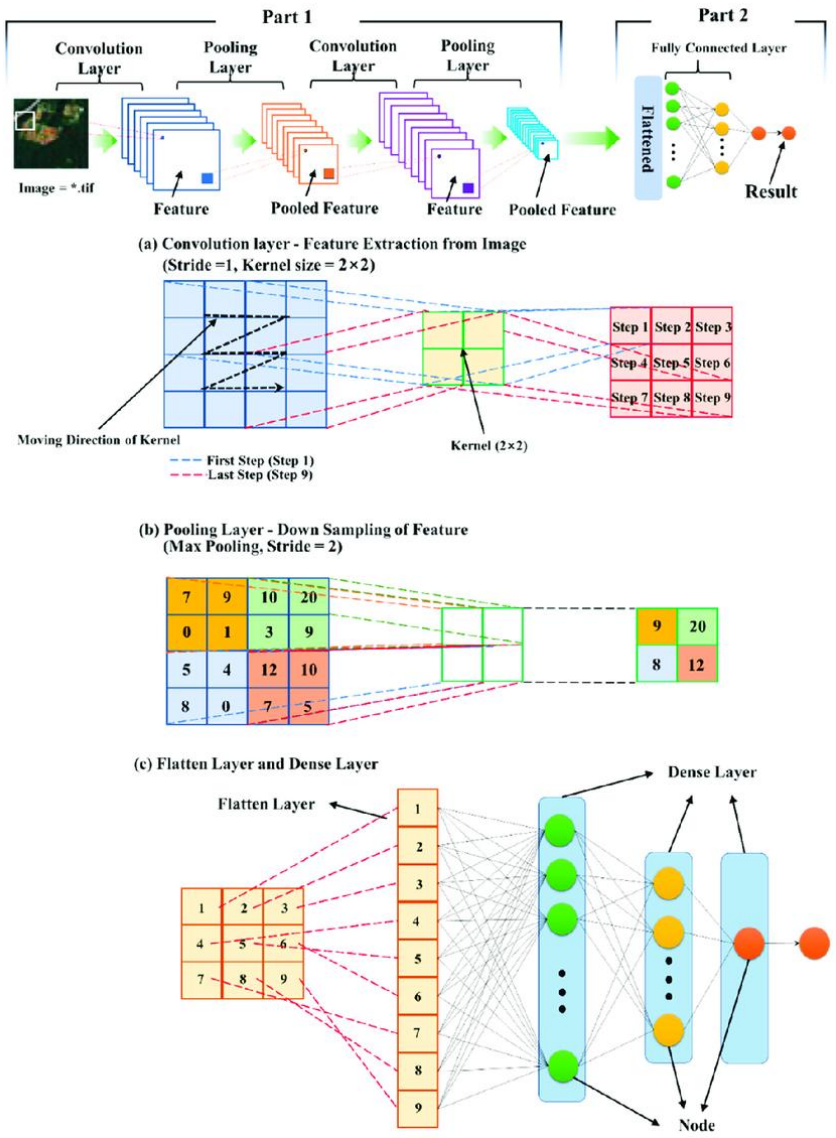
Five popular CNN Classification Architectures)

Prof Jamshid Dehmeshki

CNN backbone architecture



Five popular CNN Classification Architectures)

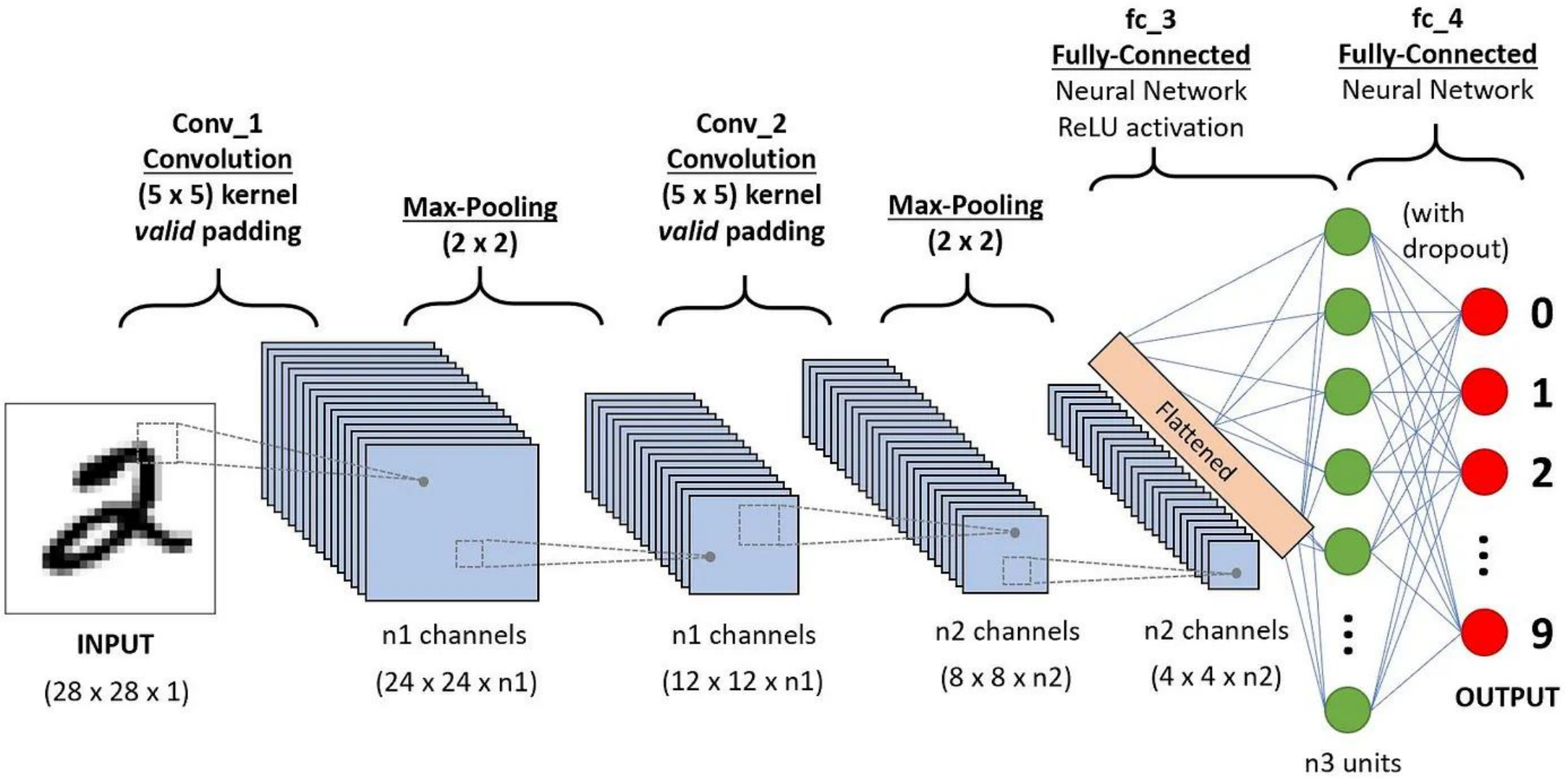
- These are the “classic” or “foundational” CNN architectures primarily designed for **image classification** tasks (such as ImageNet).

1. **LeNet**
2. **AlexNet**
3. **VGG**
4. **ResNet**
5. **Inception**

- These are often called “**backbone networks**” when used in other tasks. They all do the same **core** job:
  - given an image, they produce a feature representation that helps distinguish between classes.

1. LeNet-5 (1990s)

- Think of this as the “grandfather” of modern CNNs. It was originally used for recognising handwritten digits (like those on checks).
- It has a few convolutional layers to detect features, then some pooling layers to reduce size, and ends with a couple of fully connected layers to classify the digit (0-9).
- Very small and simple by modern standards, but pioneered many core ideas (convolution, pooling) still used in today’s deeper networks.


Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with dropout)
Flattened
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
n3 units
0
1
2
9
OUTPUT

1. LeNet-5

- **Common Modern Use Cases:**
  - Mainly as an introductory architecture in educational contexts (it's small and easy to train).
  - Still good for simple classification tasks on low-resolution images (e.g., MNIST).
- **Object Detection / Segmentation?**
  - Rarely used for larger-scale tasks, because it's shallow and not powerful enough for modern, high-resolution datasets.

2. AlexNet (2012)

- The network that famously put deep learning on the map for computer vision by winning the 2012 ImageNet competition with a big performance leap.
- Uses more filters, bigger feature maps, and deeper layers than LeNet, plus **ReLU** activations for faster training.
- **Defining Features:**
  - Introduced dropout in CNNs to fight overfitting.
  - One of the first to really leverage GPU computing to train faster on a large dataset (ImageNet).

2. AlexNet

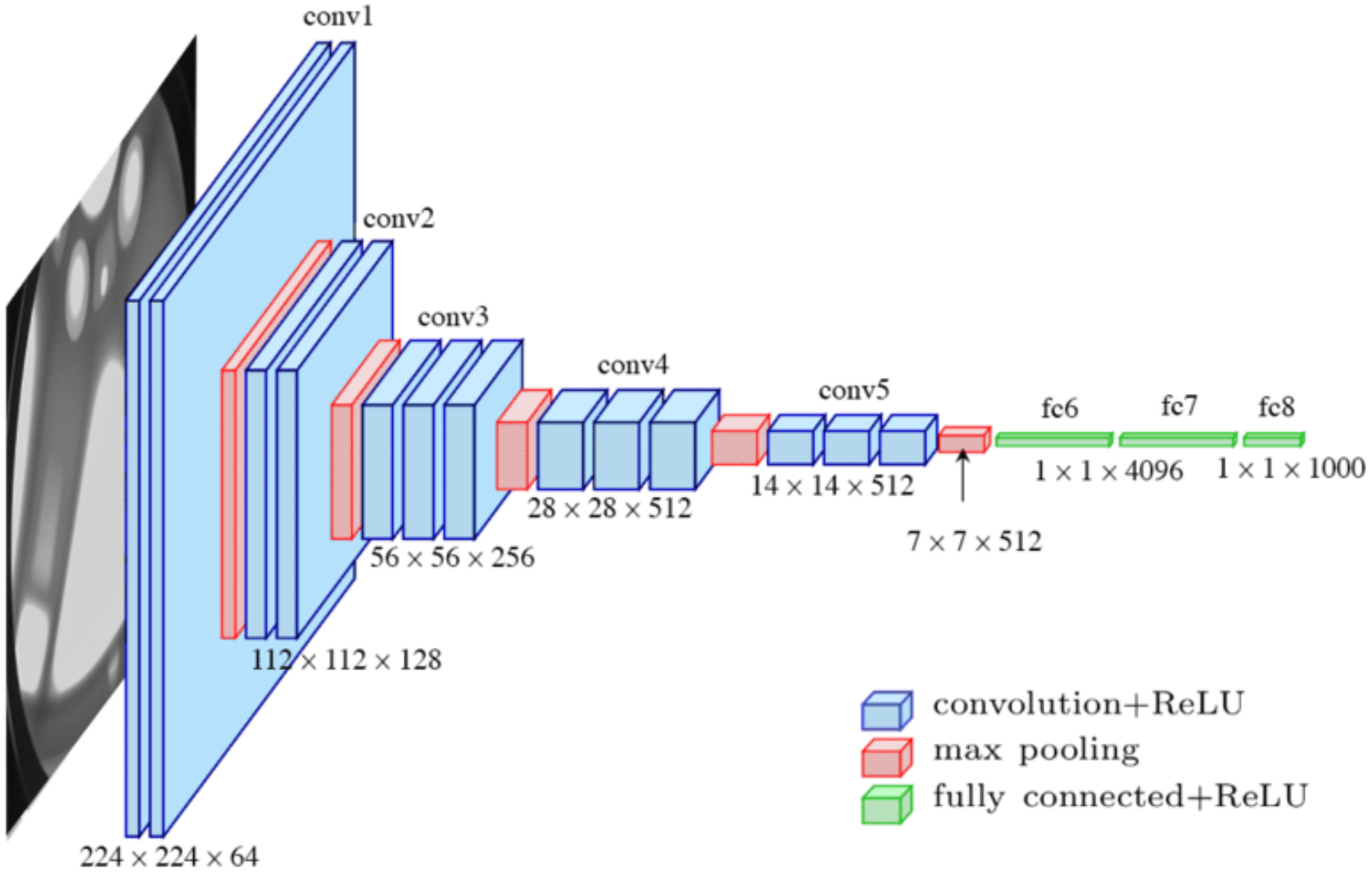
- **Originally Designed For:**
  - ImageNet classification (1,000 classes). This was its big breakthrough in 2012.
- **Common Modern Use Cases:**
  - Image Classification on moderate to large datasets.
  - Often used as a starting point to illustrate deeper CNN concepts than LeNet.
- **Object Detection / Segmentation?**
  - While not as common today, earlier detection methods like **R-CNN** sometimes used AlexNet as a backbone.
  - Now largely replaced by deeper architectures (like VGG or ResNet) for more challenging tasks.

3. VGG (2014)

- **VGG** (developed by the Visual Geometry Group at the University of Oxford) is a family of **deep** Convolutional Neural Networks typically referred to as **VGG16** or **VGG19** (indicating the number of layers).

3. VGG (2014)

- Known for its very **uniform** structure: the entire network is built mostly of 3×3 convolution filters stacked in a straightforward way.
- For example, it might stack multiple 3×3 convolutions in a row before doing a pooling step, and it does this pattern repeatedly.
- **Defining Features:**
  - Emphasised **depth** (16 or 19 layers) with simple, repeating blocks.
  - Very large in terms of parameters, making it quite demanding in terms of memory and computation.


conv1
conv2
conv3
conv4
conv5
fc6
fc7
fc8
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU

3. VGG (VGG16 / VGG19)

- **Originally Designed For:**
  - ImageNet classification with deeper stacks of **3×3** convolutions.
- **Common Modern Use Cases:**
  - A popular **feature extractor** (i.e., backbone) in many computer vision tasks.
- **Classification** on datasets where computational resources are less of an issue.
- **Transfer Learning**: Many pretrained VGG models are available for fine-tuning.
- **Object Detection / Segmentation?**
  - Widely used as a backbone for **Fast R-CNN**, **Faster R-CNN**, and **Mask R-CNN** in earlier implementations.
  - Also used in semantic segmentation frameworks like **FCN (Fully Convolutional Network)** for pixel-wise classification.

3. VGG

1. **Main Idea**:
    1. VGG stacks **multiple** small 3×3 convolution filters one after another, followed by a max pooling layer.
    2. Repeats this pattern several times, making the network **deep** while keeping the design straightforward.
2. **Why 3×3 Filters?**
    1. Using smaller filters repeatedly (rather than a single larger filter) allows the network to learn more complex features and use fewer parameters than if you used one big filter at once.
3. **Structure**:
    1. Typically, you have 2–3 consecutive 3×3 convolution layers → a pooling layer → more 3×3 conv layers → pooling → and so on.
    2. After these "convolution blocks," it often ends with **fully connected** layers for classification.
4. **Pros**:
    1. **Simplicity**: Easy to understand and implement.
    2. **Strong performance** on large-scale image classification (ImageNet).
    3. **Transfer learning**: Its straightforward architecture and large pretrained models make it popular for fine-tuning on new tasks.
5. **Cons**:
    1. **Large parameter count**: All those conv layers and fully connected layers can make VGG quite big (hundreds of millions of parameters).
    2. **Memory-heavy**: Requires more RAM/GPU memory compared to more modern, parameter-efficient networks.

- **In short, VGG showed that a deep, yet simple design—repeated 3×3 filters—could achieve high accuracy, and it became a go-to backbone for many computer vision tasks (classification, detection, segmentation) in its time.**

4. ResNet (2015)

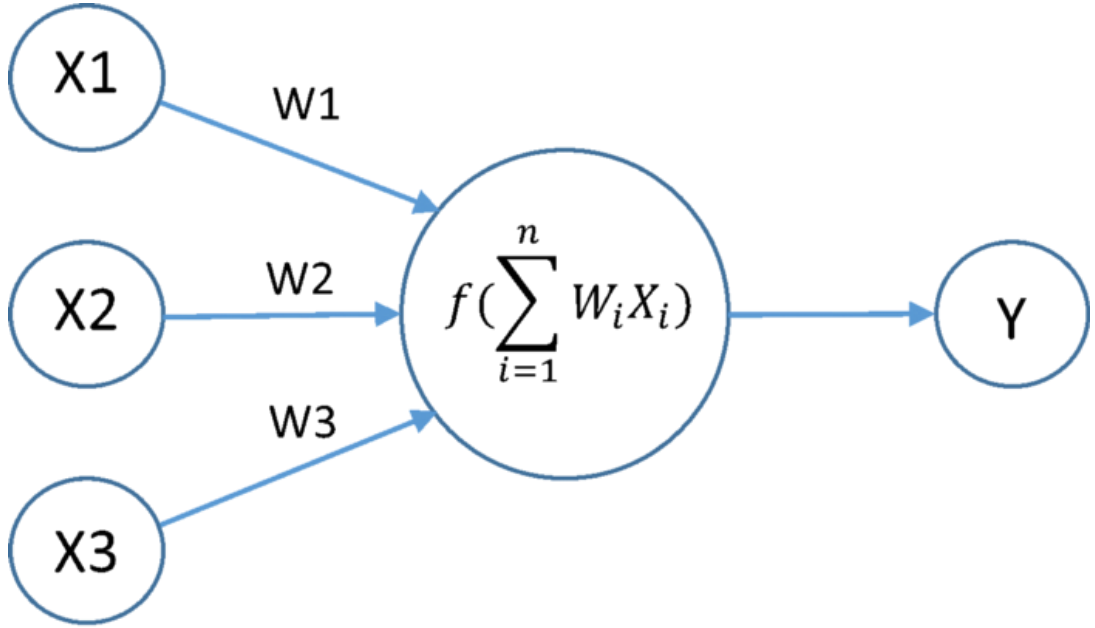
- Imagine you keep making a CNN deeper and deeper, but it gets harder to train because of vanishing gradients. ResNet solves that with **“skip connections.”**
- A skip connection is like giving the flow of information a shortcut around some layers, so the network can be super deep without training issues.
- **Defining Features:**
  - Introduced the concept of **residual blocks**, allowing over 100+ layers to be trained effectively.
  - Kick-started the trend that **deeper** CNNs can perform better, as long as they’re designed to combat vanishing gradients.

4. ResNet (2015)

- **Residual blocks** in ResNet (short for “Residual Network”) are the main innovation that allowed it to successfully train networks with **over 100 layers**.
- Think of a residual block as saying: “**Start with what we already have, then add a small tweak if necessary.**”
- Typically, in a very deep network, a problem called the **vanishing gradient** makes training extremely difficult (see my previous tutorial):
  - the gradient signals become so small by the time they reach early layers that it’s hard for the network to update those layers.

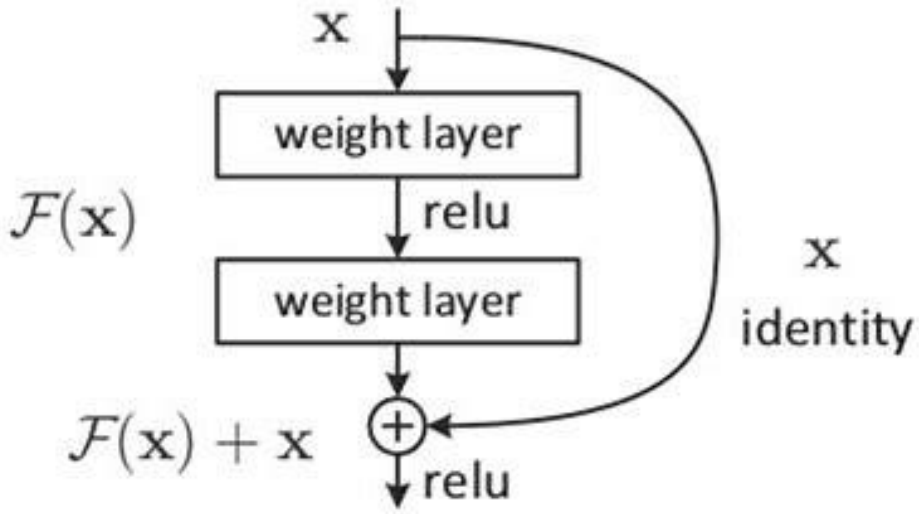
4. ResNet (2015)

- **Imagine you have a piece of text:**
  - You want to revise it to make it better.
  - Instead of rewriting the entire text from scratch, you just add corrections or improvements to it.
- **Apply that idea to a neural network layer:**
  - A regular layer takes its input (x) and tries to produce a new output (F(X) from scratch.
- A **residual block** does something different: it keeps the original input **untouched** and **adds** a small “fix” on top of it.
- Mathematically, we call the fix F(x) and the final output is x + F(x)
  - x= the original input to the block.
  - F(x) = the “improvement” or “correction” the block learns.



4. ResNet (2015)

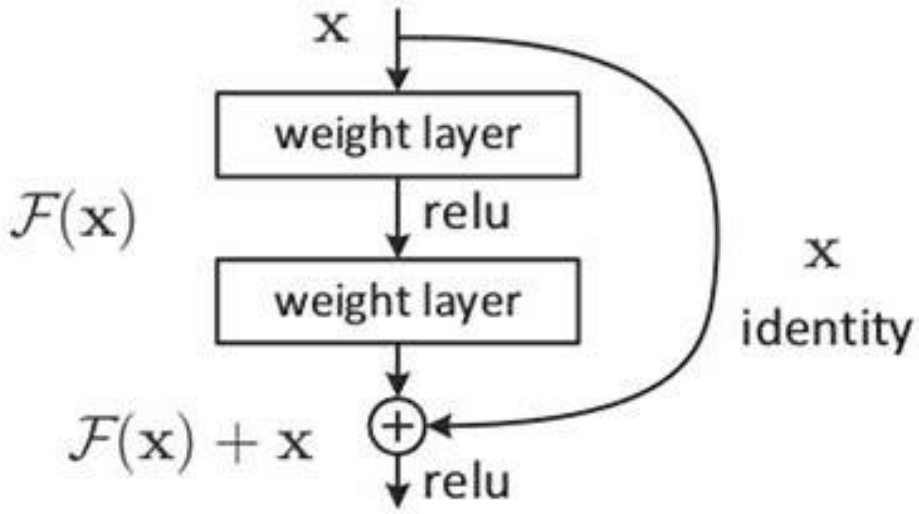
- If the block doesn't really need to change x, it can learn Y= F(x) ≈ 0 and the input just passes through unaltered.
- This means it's **much easier** for the network to learn or refine what's already there, rather than trying to reconstruct the entire feature from scratch in each layer.
- It also helps the **gradient** (the "learning signal") flow backward through the network more easily, preventing the issue where signals get too small in very deep networks.



4. ResNet (2015)

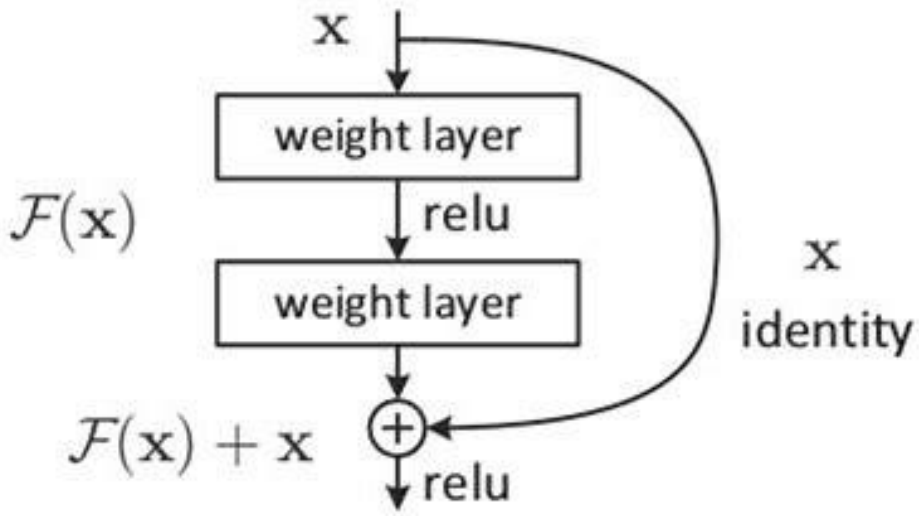
In a **residual block**, the function F(x) is simply the output of a small stack of layers (usually **convolutions** + **ReLU**) applied to the input x. Here's the typical process in plain terms:

1. **Start with the input** x
2. **Apply a series of operations**—usually:
    1. A **convolution** (e.g., 3×3 filters)
    2. A **ReLU** activation (to introduce non-linearity)
    3. (Optionally, repeat the above block one more time: Conv + ReLU)
3. The final output of these operations is your **F(x)**
4. The **residual block** output is x+F(x)
5. Sometimes there's a final ReLU after adding x+F(x)



4. ResNet (2015)

- You take your input, pass it through a couple of standard CNN layers, and that result is F(x)
- Then you add that F(x) to your original input x, effectively saying, “Here’s the original information, plus any improvements or corrections we learned”.
- This helps the network learn **residual (extra)** information without having to re-learn everything from scratch at each layer—and that’s why we call it a **“residual”** block.



4. ResNet (2015): **skip connection**

- It doesn't mean we're completely ignoring or removing those layers. Instead, there's a shortcut path that bypasses the main stack of layers in a residual block.

1. **The Regular Path (Layers in the Block)**
   - The input x goes through convolution, ReLU, batch normalization, etc. to produce F(x)
2. **The Skip / Shortcut Path**
   - A parallel path carries x **unchanged** (or with minimal changes like a 1×1 convolution if dimensions differ).

- **Final Output**
  - The **block output** is the sum of these two paths: x+F(x)
  - That is, "**original input** + **what the block learned**."
- So "**skipping**" means there's a **direct route** from the block's input to its output.
- It ensures that if the block learns no useful "correction" (F(x)≈0), the input can pass through **untouched**—making deeper networks easier to train.
- You still apply the layers in the block; the skip connection just adds a shortcut **in parallel** to them.

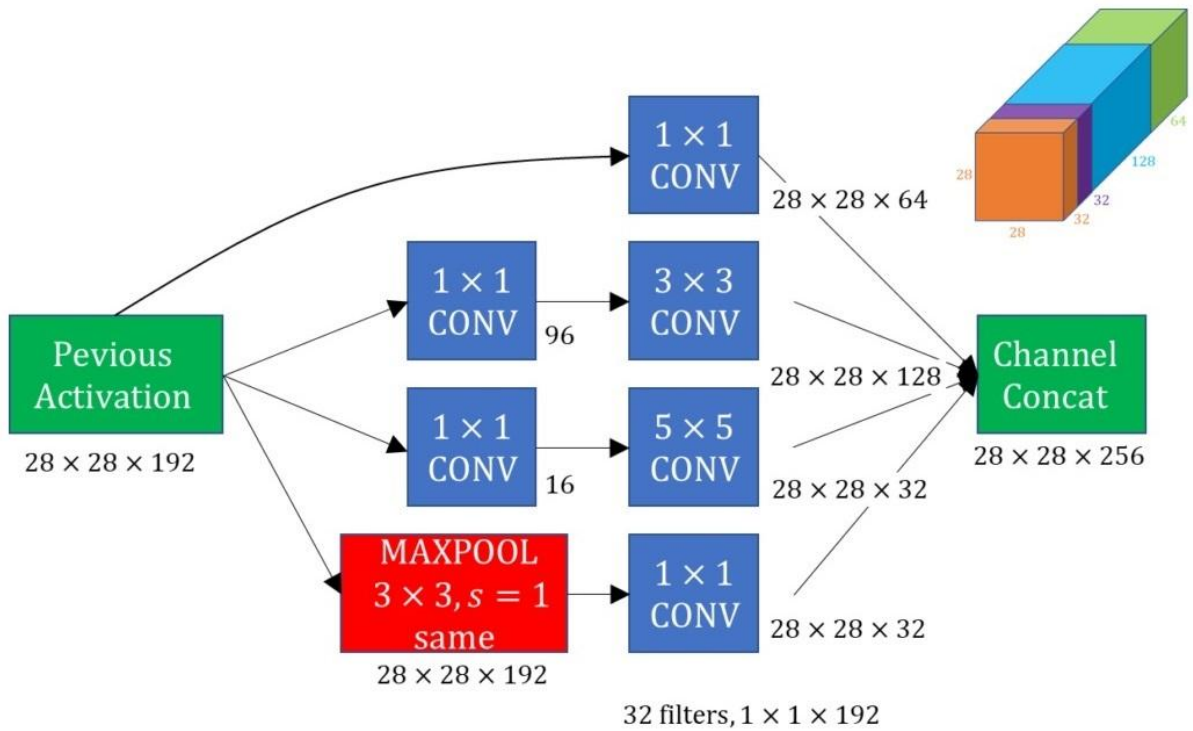
4. ResNet (ResNet-50, ResNet-101, ResNet-152, etc.)

- **Originally Designed For:**
  - Very deep ImageNet classification without vanishing gradients, thanks to **skip connections**.
- **Common Modern Use Cases:**
  - One of the **most common backbones** for a wide range of vision tasks:
    - **Classification** on large and small datasets (with different depths).
    - **Object Detection** (Faster R-CNN, Mask R-CNN often use ResNet-50/101 as their base).
    - **Image Segmentation** (e.g., UNet variants or Mask R-CNN with a ResNet backbone).
    - **Pose Estimation** (as the feature extractor in many pipelines).
- **Strength:**
  - Scales well to deeper networks (50+ layers) while remaining trainable.

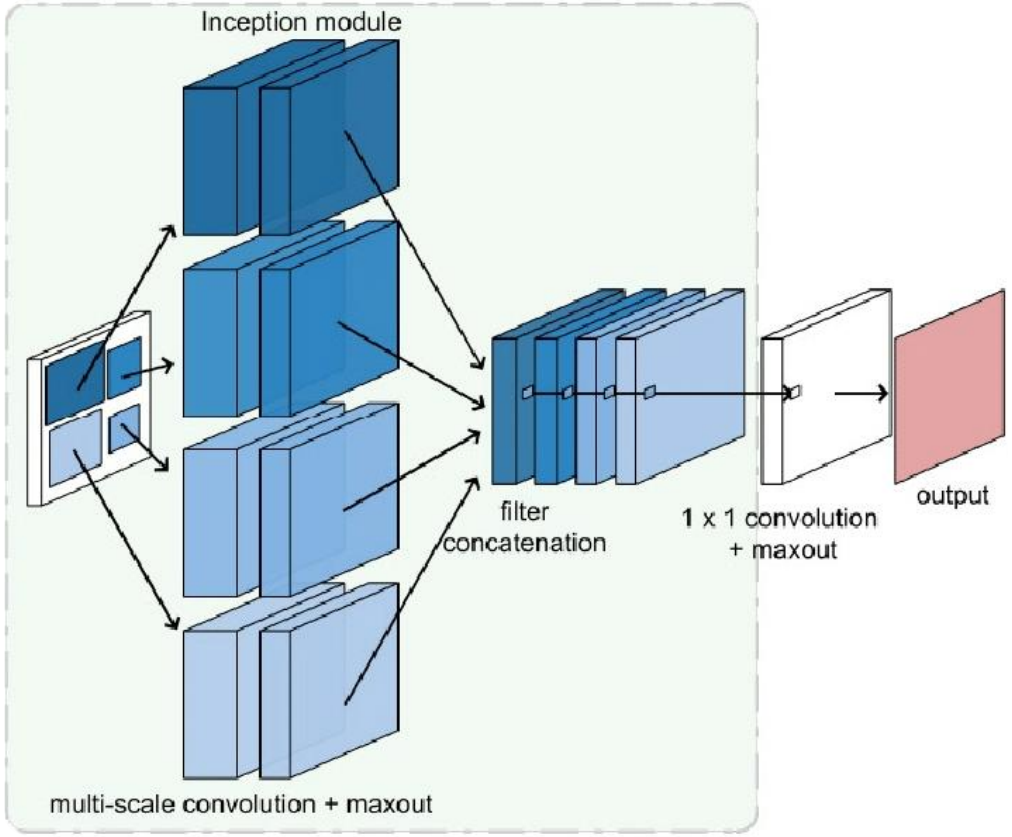
5. Inception (a.k.a. GoogleNet, 2014 onward)

- Instead of just stacking 3×3 filters, it tries **several** filter sizes (1×1, 3×3, 5×5, etc.) in parallel within a single layer and merges them.
- Think of it like a multi-path approach—some filters look for small details, others look for bigger patterns—all at once.
- **Defining Features:**
  - The **Inception modules** combine different-sized filters in a single block.
  - Uses **1×1 convolutions** smartly to reduce computation (they compress features before doing the bigger convolutions).

An example of an Inception module



5. Inception



- **Inception** (also known as **GoogLeNet** when it first appeared) is a type of Convolutional Neural Network originally designed to **improve both accuracy and efficiency** by combining different filter sizes in the same layer.

1. **Main Idea**:
    1. **"Inception modules"** let the network look at multiple scales of an image **simultaneously** (1×1, 3×3, 5×5 filters) in a **single** layer block.
    2. Instead of deciding on just one filter size (e.g., 3×3 or 5×5), the network learns to mix them, capturing both **small details** and **larger structures** at the same time.
2. **How It Works**:
    1. Each Inception module has **parallel** branches, each with a different kernel size:
        1. **1×1 convolution** (can reduce dimensionality, i.e., the number of feature maps).
        2. **3×3 convolution**.
        3. **5×5 convolution** (or 7×7 in later versions).
        4. **Pooling** (max or average).
    2. The outputs of all these branches are then **concatenated** along the channel dimension, forming a merged feature map.

5. Inception

- **Why 1×1 Convolutions?**
  - They act like **“bottlenecks”** to reduce the number of channels before doing costlier 3×3 or 5×5 convolutions.
  - This greatly cuts down the total number of parameters while still allowing the network to be deep.
- **Structure**:
  - **Stack** several Inception modules. Each module does multi-scale convolution + pooling and merges results.
  - Finally, similar to other CNNs, you have fully connected layers or global pooling + a final classification layer.

5. Inception (GoogLeNet, Inception v3, Inception-ResNet, etc.)

- **Originally Designed For:**
  - ImageNet classification with **Inception modules** (parallel filters of different sizes).
- **Common Modern Use Cases:**
  - **Classification** on large datasets with efficient resource usage (the 1×1 “bottleneck” layers help keep parameter counts lower).
  - Variants like **Inception-ResNet** combine skip connections with Inception modules for even deeper networks.
  - **Object Detection / Segmentation?**
    - Used in Google’s **“Slim” or “TensorFlow”** detection models. Some detection frameworks have used Inception as a backbone.
    - Can be adapted for **segmentation** or **detection**, though ResNet-based backbones are often more popular in open-source frameworks.

5. Inception

1. **Pros**:
    1. **Better resource efficiency** than a naive wide network.
    2. **Multi-scale feature extraction** in each module (the network can learn both fine and coarse features in parallel).
    3. Achieved very strong performance on ImageNet and COCO detection tasks.
2. **Cons**:
    1. Can be **more complex** to implement. Each Inception module has multiple branches that need to be combined.
    2. Later versions (e.g., **Inception-v3**, **Inception-v4**, **Inception-ResNet**) introduced additional tweaks that can make the architecture more intricate to set up.

- **Bottom Line**:
  Inception networks are about **parallel multi-scale filters** in each layer. They use **1×1 convolutions** as clever "bottlenecks" to keep the model efficient, while still allowing the network to learn both **fine** and **wide** features simultaneously.

Summery

1. Five Popular CNNs (Classification Architectures)

These are the “classic” or “foundational” CNN architectures primarily designed for **image classification** tasks (such as ImageNet). Examples:

1. **LeNet**
2. **AlexNet**
3. **VGG**
4. **ResNet**
5. **Inception**

- These are often called “**backbone networks**” when used in other tasks. They all do the same **core** job: given an image, they produce a feature representation that helps distinguish between classes.

Key Points About Their Applications

1. **Classification**
    1. **All** these architectures were fundamentally designed for image classification (especially on ImageNet).
    2. You can directly use them for classification or fine-tune them for new classes via **transfer learning**.
2. **Object Detection and Segmentation**
    1. Modern detection/segmentation methods (like **Faster R-CNN**, **Mask R-CNN**, **YOLOs**, **UNet**, etc.) need a strong **feature extractor** (called the “backbone”).
    2. **VGG**, **ResNet**, and **Inception** (and their variants) are commonly used backbones because they learn robust, hierarchical features.
    3. **ResNet** is particularly popular because it can scale deeply, giving excellent performance.
3. **Computational Efficiency and Deployment**
    1. Some networks (VGG) have very large parameter counts, making them heavier for real-time applications.
    2. Others (ResNet, Inception) are more parameter-efficient and can handle deeper layers.
    3. In practice, one often uses **lighter variants** (e.g., MobileNet, ResNet-18) for on-device tasks.

Comparison

- **LeNet**: Historical, small-scale classification.
- **AlexNet**: Early deep CNN success on ImageNet classification.
- **VGG**: Very deep, uniform 3×3 convs; popular backbone in older detection/segmentation but big parameter count.
- **ResNet**: Deep networks with skip connections; extremely common backbone for detection and segmentation.
- **Inception**: Multi-branch convolutions of different sizes; also widely used in classification and extended for detection/segmentation in some frameworks.
- All can be adapted (with varying degrees of modification) for tasks like **object detection** and **image segmentation**, but in modern practice, **ResNet** (and sometimes **Inception** or **Inception-ResNet**) are the most widespread backbones due to their depth, performance, and ease of integration into state-of-the-art vision pipelines.

Comparison

- **Depth and Width:**
  - LeNet is shallow, AlexNet is moderately deep, VGG is deeper but with uniform 3×3 filters, ResNet can get extremely deep due to skip connections, and Inception has a unique multi-branch structure in each layer.
- **Parameter Efficiency:**
  - VGG is known to be parameter-heavy, while Inception introduced ways to reduce the number of parameters (via 1×1 conv “bottlenecks”). ResNet also is more parameter-efficient compared to older architectures of similar depth.
- **Performance and Training:**
  - Over time, architectures became **deeper** and more sophisticated in how they handle information flow, which generally improved accuracy on large datasets like ImageNet.
- **Practical Deployment:**
  - Simpler networks (like smaller ResNets or MobileNet, a later architecture designed for mobile/embedded devices) are more common in real-world applications due to **speed and memory** constraints.
  - Very large networks like VGG or large ResNets can be used on big servers for high-accuracy tasks but are often too bulky for smaller devices.